UFL + FEniCS: DSL Case Study

Russell Bentley

Stony Brook University

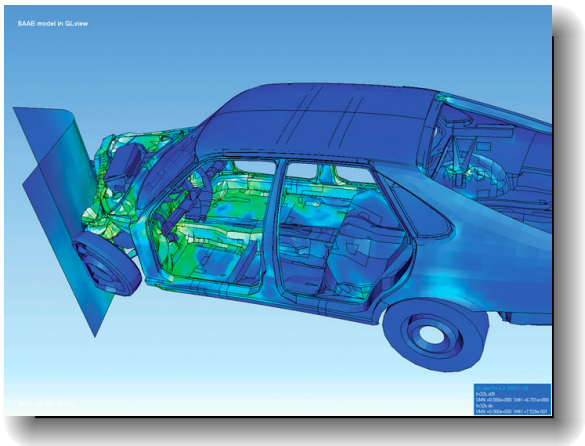
September 18 2025

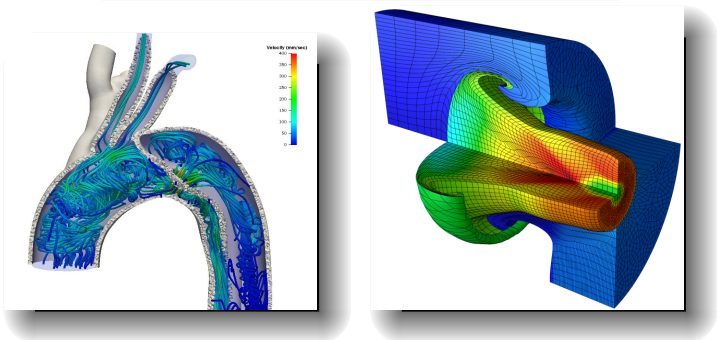
What are we talking about today?

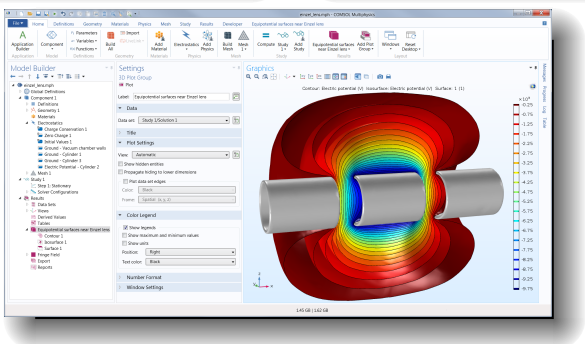
- Finite Element Method (FEM)
- FEniCS
- Unified Form Language (UFL)
- Discussion Questions (?)

Finite Element Method (FEM)?

- Family of numerical methods
  - Solve PDEs
  - Mathematical Modelling
  - Engineering / Science
- History
  - Structural Mechanics in 1940s
  - NASTRAN in 1960s
  - Widely available now
- Broad and well-funded

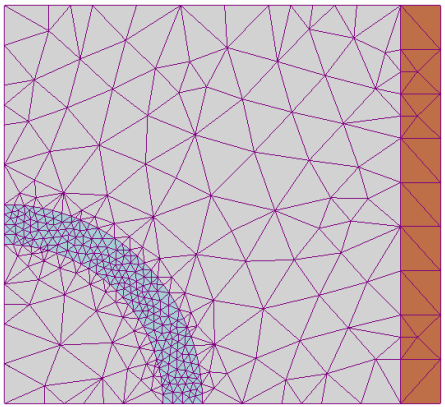




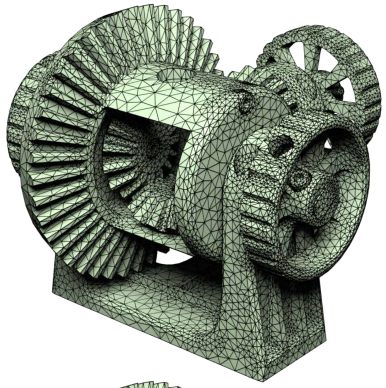


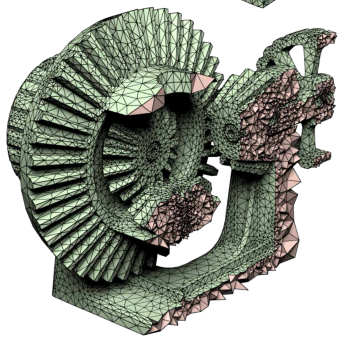
Images from Wikipedia, Readfast, LNLL, COMSOL

FEM: Geometry and Meshing



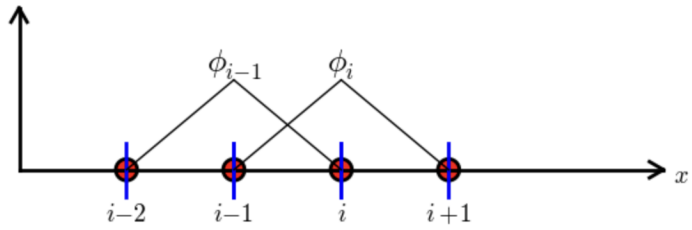
- Domain discretized with mesh
  - Volumetric elements
  - Node connectivity
- Bottleneck in production
- Massive research topic
- Meshing Out-of-scope for today

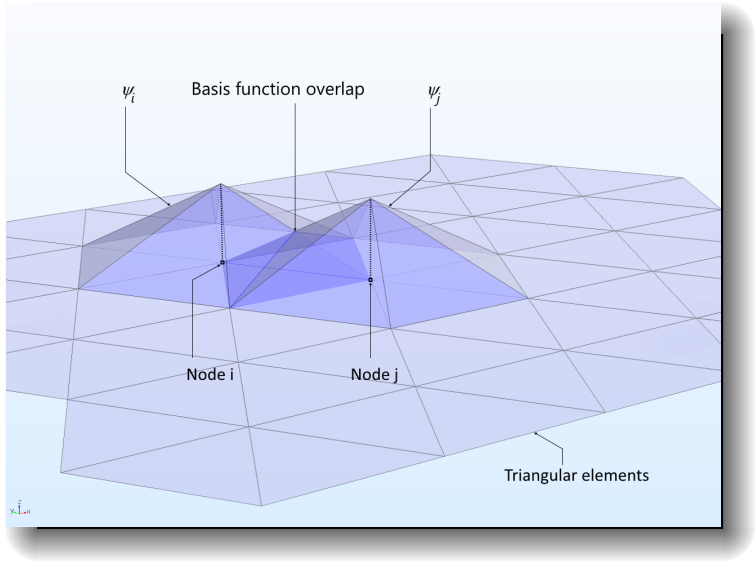




FEM: Basis Functions

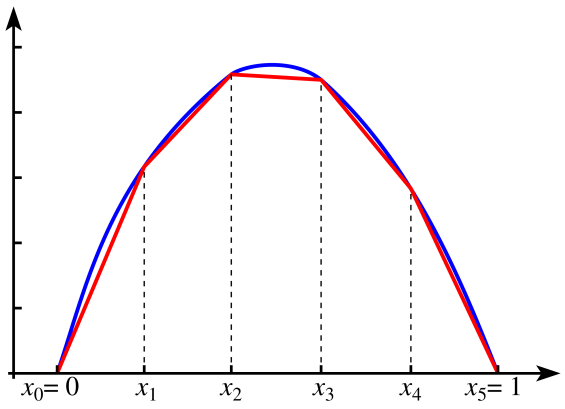
- Mesh defines a function space
- Basis functions ϕ_i
  - One per mesh node
  - Local support
- Used for
  - Computation
  - Representing the solution
- $u(x) = \sum_{j=1}^{N} U_j \phi_j(x)$





FEM: Weak Formulation

- Derive solution from trial function space
- Requiring equality everywhere is impossible
- Relaxed continuity restrictions
- Assert that residual (error) is orthogonal to *test function* space
- Usually use ϕ_i from mesh

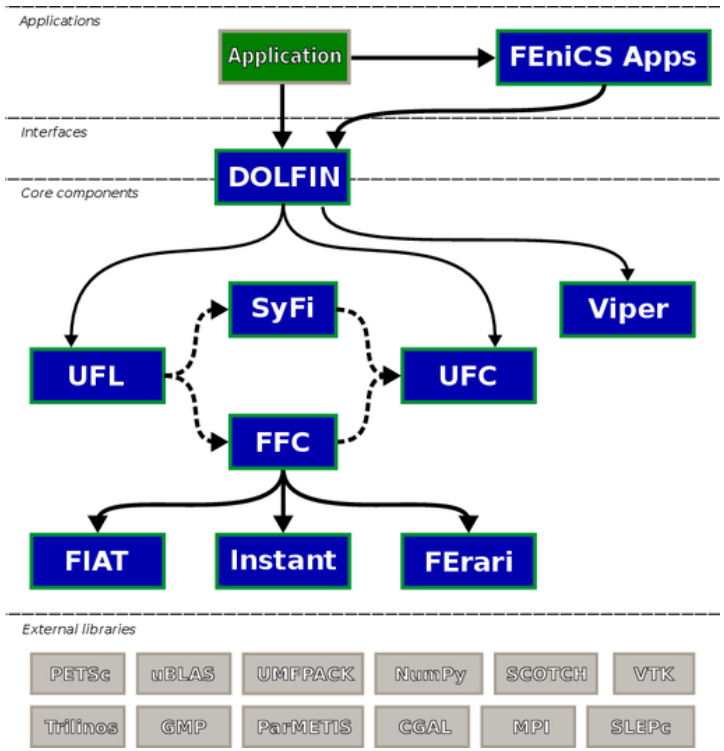


Strong Form: $L(u) = f$ on Ω

Variational Form: $\int_\Omega (L(u) - f)v = 0$

The FEniCS Project

- Collection of opensource software
- Used to create models with FEM
- Origins in 2002
- Steady development / rewrites
- Very popular for research
- Unified Form Language (UFL)



Unified Form Language (UFL)

- Introduced in 2013
  - Grew out of FEniCS Form Compiler (FCC)
- DSEL in Python
- Writing equations
  - Variational Forms
  - Suitable to solve with FEM
  - Syntax closely matches notation
- Adopted by other projects

```
a = c * dx + e * ds
```

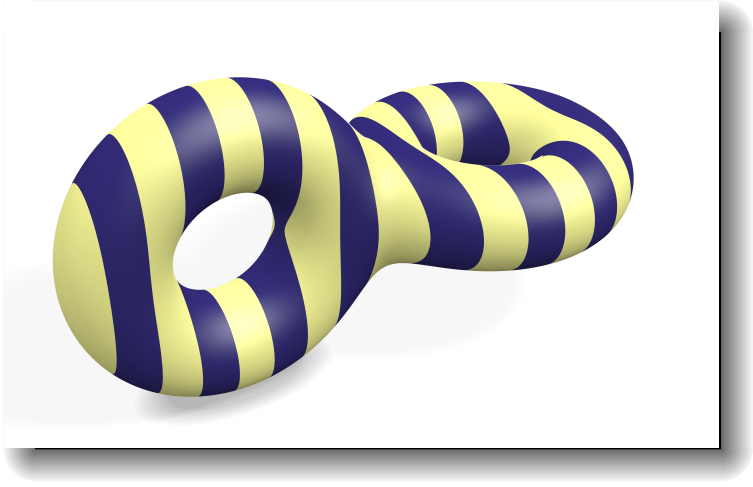
represents

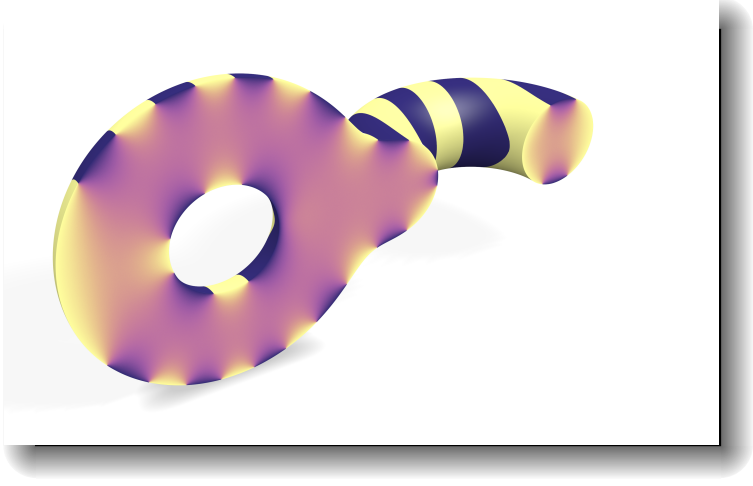
$$a = \int_\Omega c \; dx + \int_{\partial\Omega} e \; ds$$

Poisson's Equation: Hello World of PDEs

$$-\nabla^2 u = -\Delta u = f \text{ in } \Omega$$

- Constain Divergence of Gradient
- Ubiquitous
  - Thermal Equilibrium
  - Electrostatics
  - Gravitational Potential
  - Vector Graphics Shading
  - Component of larger problems
- Solve for u





Variational Form for Poisson's Equation

$$-\Delta u = f \text{ in } \Omega$$

Multiply both sides by a *test function* v
Integrate over the domain Ω

$$\int_\Omega (-\Delta u) v \; dx = \int_\Omega f v \; dx$$

Apply integration by parts, test function disappears on the boundary.

$$\int_\Omega \nabla u \cdot \nabla v dx = \int_\Omega f v \; dx$$

Variational Form Find $u \in V$ such that

$$a(u, v) = L(v) \; \forall v \in \hat{V},$$

Variational Form for Poisson's Equation in UFL

```
from ufl import *

# Mesh and function space
# Provided by higher-level FEniCS code
V = FunctionSpace(mesh, "Lagrange", 1)

u = TrialFunction(V) # unknown
v = TestFunction(V)  # test function
f = Function(V)      # source term

a = dot(grad(u), grad(v)) * dx
L = f * v * dx
```

Poisson's Equation: The Discrete Problem

Solution is weighted sum of basis functions

$$u(x) = \sum_{j=1}^{N} U_j \phi_j(x)$$

Variational Form becomes

$$\sum_{j=1}^{N} U_j \int_\Omega \nabla \phi_j \cdot \nabla \hat{\phi}_i \; dx = \int_\Omega f \hat{\phi}_i \; dx \;\; i = 1, 2, \cdots, N$$

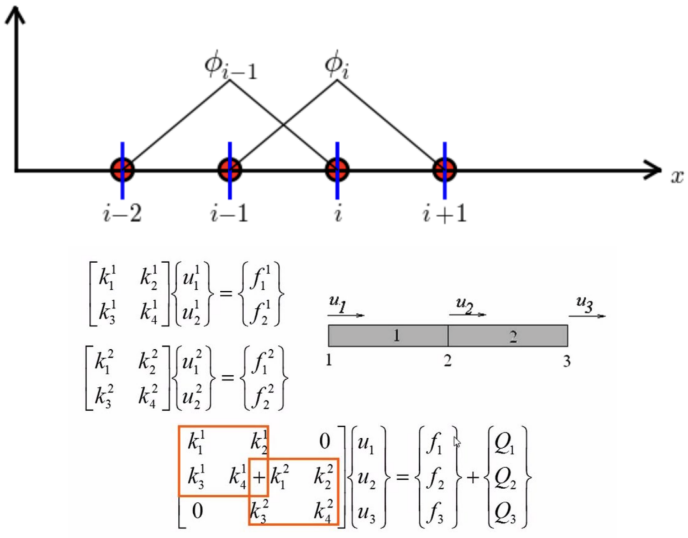
Compute U by solving linear system $AU = b$, where

$$A_{ij} = \int_\Omega \nabla \phi_j \cdot \nabla \hat{\phi}_i \; dx$$

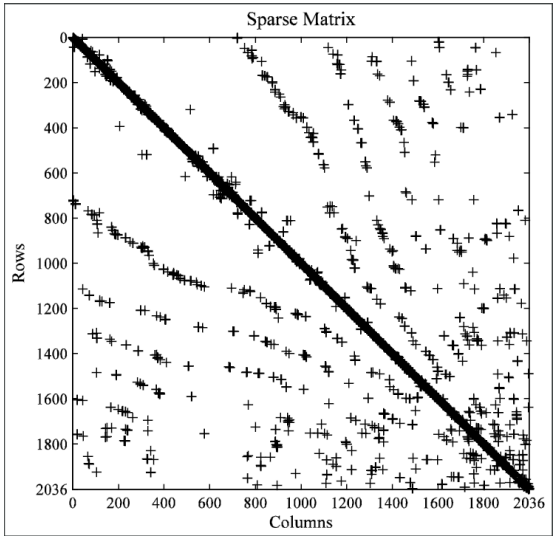
$$b_i = \int_\Omega f \hat{\phi}_i \; dx$$

Global Assembly and Linear Solve

$$A_{ij} = \int_{\Omega} \nabla \phi_j \cdot \nabla \hat{\phi}_i \; dx$$
$$= \sum_{\{E\}} \int_{\Omega_E} \nabla \phi_j \cdot \nabla \hat{\phi}_i \; dx$$

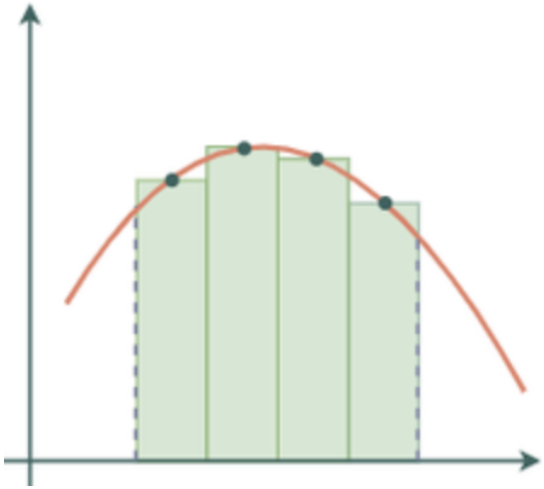


- A_{ij} is non-zero when ϕ_i and ϕ_j share an element
- A matrix is sparse
- Use a sparse linear solver



Integrating with Quadrature

- Integrals mean quadrature rules
  - $\{(w_k \in \mathbb{R}, x_k \in \Omega)\}$ such that $\sum_{k=1}^{K} w_k = 1$
- Integrators have nested structure
  - For each element
  - For each basis pair
  - For each quadrature point



$$A_{ij} = \int_{\Omega_E} \nabla \phi_j \cdot \nabla \hat{\phi}_i \, dx$$
$$\approx \sum_{k=1}^{K} w_k \nabla \phi_j(x_k) \cdot \nabla \hat{\phi}_i(x_k)$$

Coding FEM by Hand

```cpp
/// Used in thermal scenarios to integrate element conductivity
class ThermalConductivity : public BilinearForm<1, ThermalConductivity> {
public:
  ThermalConductivity(QuadratureRule const& real_rule, size_t n_dofs,
                      BasisValuesCoef const& basis_values_coef,
                      Coef<double> const& conductivity_coef)
      : BilinearForm(real_rule, n_dofs, basis_values_coef),
        m_conductivity_coef(conductivity_coef) {}

  vec1 integrate(size_t basis_index_i, size_t basis_index_j) const {
    Eigen::Matrix<double, 1, 1> result = Eigen::Matrix<double, 1, 1>::Zero();
    for (size_t qp = 0; qp < m_real_rule.size(); qp++) {
      double const conductivity = m_conductivity_coef.value(qp);
      auto const& basis_value_i =
          m_basis_values_coef.value(qp, basis_index_i, 0);
      auto const& basis_value_j =
          m_basis_values_coef.value(qp, basis_index_j, 0);
      vec3 ri_0 = basis_value_i.derivatives();
      vec3 rj_0 = basis_value_j.derivatives();

      auto const per_dim_contrib =
          m_real_rule[qp].weight * ri_0.transpose() * rj_0 * conductivity;
      result += per_dim_contrib;
    }
    return result;
  }

private:
  Coef<double> const& m_conductivity_coef;
};
```

Unified Form Language (UFL)

- *Focused* on multilinear forms
  - Linear in argument spaces $\{V_j\}_{j=1}^{\rho}$
  - Parameterized by coefficent spaces $\{W_k\}_{k=1}^{n}$

$$a : W_1 \times \cdots \times W_n \times V_1 \times \cdots \times V_\rho \to \mathbb{R}$$

- Expression based
  - Terminals and operators
  - Tensor shape as type

Table IV. Tables of Nonliteral Terminal Expressions

Geometric quantities		*Functions*	
Math.	UFL notation	Math.	UFL notation
x	`x = cell.x`	$c \in \mathbb{R}$	`c = Constant(cell)`
n	`n = cell.n`	$g \in V$	`g = Coefficient(element)`
$\|T\|$	`h = cell.volume`	$w \in \mathcal{V}$	`w = Argument(element)`
$r(T)$	`r = cell.circumradius`	$u \in \mathcal{V}$	`u = TrialFunction(element)`
$\|F\|$	`fa = cell.facetarea`	$v \in \mathcal{V}$	`v = TestFunction(element)`
$\sum_{F \subset T} \|F\|$	`ca = cell.cellsurfacearea`		

Math. notation	UFL notation
$A_{,i}$ or $\frac{\partial A}{\partial x_i}$	`A.dx(i)` or `Dx(A, i)`
$\frac{dA}{dn}$	`Dn(A)`
$\operatorname{div} A$	`div(A)`
$\operatorname{grad} A$	`grad(A)`
$\nabla \cdot A$	`nabla_div(A)`
$\nabla A \equiv \nabla \otimes A$	`nabla_grad(A)`
$\operatorname{curl} A \equiv \nabla \times A$	`curl(A)`
$\operatorname{rot} A$	`rot(A)`
$v = e$	`v = variable(e)`
$\frac{dA}{dv}$	`diff(A, v)`
$\mathrm{d} f$	`exterior_derivative(f)`

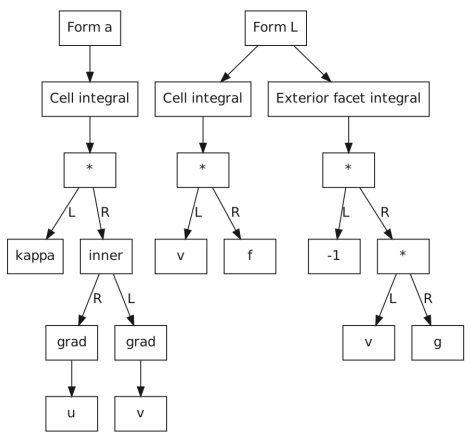


Fig. 1. Expression trees for the H^1 discretization of the Poisson equation (21).

UFL Applications

- Write your equations in math-like notation
- Get an efficient solver
- General purpose
  - Cardiac modeling
  - Plasma Physics
  - Fracture Mechanics
  - Differentiable Physics
  - Parameter Estimation
  - Optimization

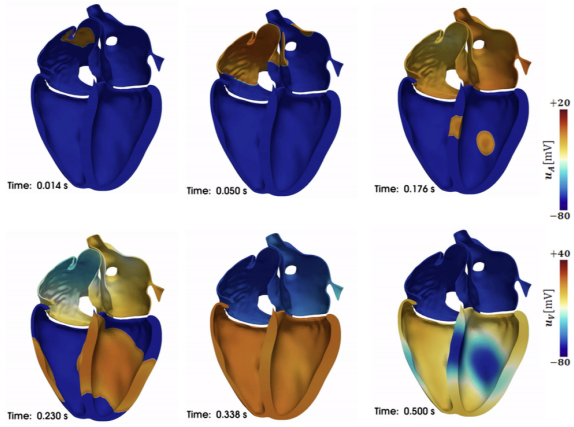
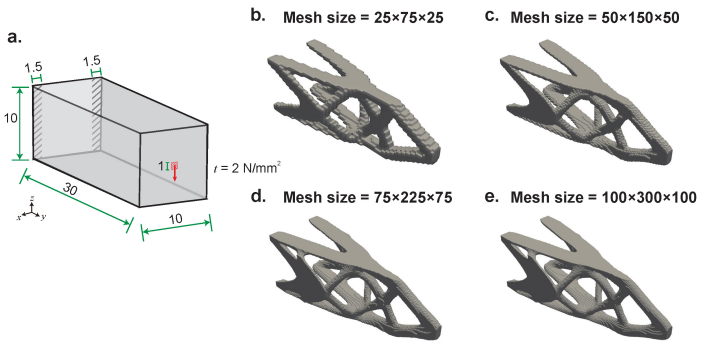


Fig. 8: Propagation of the action potential in a whole heart simulation (credits: R. Piersanti, Politecnico di Milano.



Stony Brook University

Conclusion

- UFL
  - Tightly focused language
  - Developed and used by practitioners
- Discussion
  - Interest to CS folks?